



FOR SOFTWARE ENGINEERS: ASSERTIONS ON REAL REFUNDS

JUnit vs AssertJ vs Hamcrest

What the failure tells you

The fluent syntax was never the value. The richer vocabulary is.

spring-boot-starter-test puts JUnit, AssertJ and Hamcrest on your classpath. All three export `assertThat`, and most people never chose between them.

Most of the time it does not matter, and this sheet starts by proving that. Then it goes after the cases where it does: collections, failure paths, several observations of one outcome, and coverage.

Hamcrest release dates are stated as at September 2026.

In scope

- What each library prints when an assertion fails
- Asserting on a failure path: type, message and data
- Soft assertions, and where they stop being useful
- What coverage can and cannot tell you

Out of scope

- Rewriting simple JUnit assertions for stylistic purity
- Any claim that Hamcrest is dead

The recommendation is not "use AssertJ everywhere". It is: use the vocabulary that states the invariant.

01 One failure, three libraries

Scalar, expected 4000 got 3500	all three print the same thing
Collection, JUnit	both lists, the diff left to your eyes
Collection, Hamcrest	names the index: item 2 was "k3"
Collection, AssertJ	names what was missing and what was not expected
assertTrue(...anyMatch...)	expected: <true> but was: <false>
Coverage, weak test vs precise test	the same 9 lines hit, the same 5 missed

Real messages from real runs.

02 Make the test say what it checks

A boolean assertion forces the reader to run the lambda in their head, and when it fails it can only say true was false. Extracting a property and stating the exact sequence makes three claims: which property, what order, how many.

WeakVersusPreciseTest.java: weak

```
@Test
@DisplayName("weak: a boolean that says nothing about what was expected")
void weakBooleanAssertion() {
    // The reader has to mentally execute the lambda to know the claim.
    assertTrue(ORDER.attempts().stream()
        .anyMatch(a → a.amountCents() = 5_000));
}
```

WeakVersusPreciseTest.java: precise

```
@Test
@DisplayName("precise: names the property, the order, and the cardinality")
void preciseExtractingAssertion() {
    assertThat(ORDER.attempts())
        .extracting(RefundAttempt::amountCents)
        .containsExactly(6_000L, 5_000L, 1_000L);
}
```

The precise version reports the missing 5000 and the unexpected 2000 separately.

Measured on the machine that built this sheet, not quoted

03 A failure path has three parts of contract

`assertThrows` is green for any rejection, including the wrong one. Assert the type AND the message a support agent or API client reads, or assert on the throwable itself. The message is the contract when a person reads it; typed data is the contract when a caller branches on it.

● ● ● TddAnExceptionTest.java: green for the wrong reason

```
@Test
@DisplayName("weak: green for ANY refund rejection, including the wrong one")
void weakExceptionAssertion() {
    // Passes whether the refusal is about the balance, the amount, the
    // capture, or a replay misdiagnosed as any of those.
    assertThrows(RefundRejected.class,
        () → policy.refund(ORDER, "k9", 6_000));
}
```

● ● ● TddAnExceptionTest.java: the type and the message

```
@Test
@DisplayName("precise: the type AND the message the caller actually reads")
void preciseExceptionAssertion() {
    assertThatExceptionOfType(RefundRejected.class)
        .isThrownBy(() → policy.refund(ORDER, "k9", 6_000))
        .withMessage("refund 6000 exceeds captured 4000 on A-1");
}
```

The precise assertion failed with "Expecting message to be exceeds captured, but was exceeds refundable", naming `RefundPolicy.java` line 35.

Measured on the machine that built this sheet, not quoted

04 Soft assertions: several observations of ONE outcome

Hard assertions stop at the first failure, so you learn one thing per run. `assertSoftly` reports every failure in one run, each labelled with `as(...)`. It is for three observations of one refund, not for unrelated requirements sharing a method.

SoftAssertionsTest.java

```
@Test
@DisplayName("soft: one run, every failure, all three observations of one refund")
void softAssertionsReportEverything() {
    RefundResult result = policy.refund(ORDER, "k2", 3_000);

    SoftAssertions.assertSoftly(softly → {
        softly.assertThat(result.refundedCents())
            .as("amount refunded").isEqualTo(4_000);
        softly.assertThat(result.order().refundableCents())
            .as("balance left").isEqualTo(1_000);
        softly.assertThat(result.order().attempts())
            .as("attempt history").hasSize(3);
    });
}
```

05 Coverage says what ran, never what was checked

Both tests execute the same lines of the refund method. One would notice a wrong refund amount; the other would not. The gutter is the same colour for both.

CoverageSaysWhatRanTest.java

```
@Test
@DisplayName("100% of the lines, 0% of the meaning")
void weakAssertionWithFullCoverage() {
    RefundResult result = policy.refund(Order.captured("A-1", 10_000), "k1", 3_000);

    // Exercises every line of the happy path. Checks nothing about refunds.
    assertNotNull(result);
}

@Test
@DisplayName("the same lines, and an assertion that would notice a wrong answer")
void preciseAssertionWithTheSameCoverage() {
    RefundResult result = policy.refund(Order.captured("A-1", 10_000), "k1", 3_000);

    assertThat(result.refundedCents()).isEqualTo(3_000);
    assertThat(result.order().refundableCents()).isEqualTo(7_000);
}
```

Lines 19, 22, 23, 26, 29, 32, 33, 37 and 38 covered, lines 24, 27, 30, 34 and 35 missed, identically for both tests.

Measured on the machine that built this sheet, not quoted

06 Where that leaves Hamcrest

As at September 2026, Hamcrest 3.0 shipped in August 2024, five years after 2.2. Pace is not the argument: a mature library does not need monthly releases. The question is whether matcher composition gives the clearest test for your domain. Sometimes it does, especially where an API hands you matchers. Often the fluent version reads more plainly.

Hamcrest release history, checked September 2026.

Maven Central, `org.hamcrest:hamcrest`

Before you trust a green test

1. Does this assertion say what it checks, or only that something was true?
2. On a failure path, is it asserting the type, the message and the data?
3. Could this test be green for the wrong rejection?
4. Are these soft assertions observations of one outcome?
5. Would anything here notice the method returning the wrong number?
6. Is this simple enough that plain JUnit reads better?

Everything else

 Site https://code-with-sam-dev.github.io	 Facebook https://www.facebook.com/codewithsamdev
 YouTube https://www.youtube.com/@CodewithSam-Dev	 GitHub https://github.com/code-with-sam-dev
 LinkedIn https://www.linkedin.com/company/code-with-sam-dev	 Buy me a coffee https://buymeacoffee.com/codewithsam
 x https://x.com/CodeWithSamDev	 Buy me sadza https://buymesadza.com/codewithsam
 TikTok https://www.tiktok.com/@codewithsamdev	Runnable code https://github.com/code-with-sam-dev/assertions-demos
 Instagram https://www.instagram.com/codewithsamdev	

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which of these you have actually shipped.

Java and OpenJDK are trademarks or registered trademarks of Oracle and/or its affiliates. Eclipse Temurin is a trademark of the Eclipse Foundation. They appear here only to identify the technology being discussed. Nothing here implies endorsement by, or affiliation with, any of them.

Technical claims verified 2026-09-15 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.